# KlintCode Private Agent

Autonomous Code Generation on AMD Hardware

Track 2: Private AI Agents

# Project Background & Vision

## Data Security Dilemma

Enterprise-grade software development cannot tolerate the risk of leaking proprietary logic, intellectual property, or confidential algorithms to public cloud LLMs.

## Context Fragmentation

Traditional coding assistants struggle with complex architectural tasks due to limited context windows, resulting in logical gaps across decoupled files.

**Solution:** A fully local, air-gapped autonomous agent framework running natively on AMD hardware, capable of end-to-end microservice generation.

# System Architecture (M1 - M5 Pipeline)




**LLM provides intelligence; KlintCode controls the process; TeaQL guarantees deterministic output.**

# AMD Hardware Optimization

- **Hardware:** AMD Radeon PRO W7900 48GB
- **Engine:** ROCm-enabled llama.cpp
- **Model:** Qwen3.6-35B-A3B GGUF Q4_K_M
- **Context:** 262,144 tokens, one inference slot

## Key Optimizations

Full GPU layer offloading (**-ngl 99**), quantized MoE weights, and single-slot KV-cache allocation (**-np 1**).

**20.4GB**
Model weights

**~26GB**
VRAM in long context

**88-92 t/s**
Generation

**~1,605 t/s**
40K prompt processing

# Core Technical Mechanisms

**1. Burn-After-Reading Context Management**
Dynamically purges validated intermediate code and execution logs from the prompt history, retaining only crucial metadata. This prevents context overflow and maintains inference speed during massive tasks.

**2. DAG-Based Schema Validation**
Validates inter-model dependencies locally as a Directed Acyclic Graph (DAG) to prevent circular dependencies and isolated endpoints before compilation.

**3. Dynamic Temperature Scaling**
Progressively increases the LLM temperature parameter during the M4 repair loop, effectively preventing the agent from getting stuck in local optima during syntax corrections.

# Business Benchmark Validation

## Target Scenario

Moving Company Backend Service

Generated from a zero-shot text description outlining vehicles, personnel, finance, and operations.

## Checked-in Evidence

81 Enterprise Domain Objects

10 included XML model files + main.xml

TeaQL deterministic validation gate

Rust generation + local CPU compiler check

No source code sent to external AI services

# Summary & Future Roadmap

- **Practical Value:** Drastically reduces the time and cost of private, enterprise-grade codebase initialization without compromising data sovereignty.
- **Language Expansion:** Planned support for generating Java (Spring Boot) and Go architectures via the TeaQL ecosystem.
- **Dynamic Sharding:** Implementing intelligent memory sharding to support various AMD GPU VRAM specifications for broader developer access.

## Contact Information

**GitHub:** @philipgreat | **Email:** philip_chang@163.com